

Executive Summary

We evaluated three open-source repositories—**mmore** (EPFL LiGHT), **RAG-Anything** (HKU), and **Docling** (IBM)—for solving a multimodal Retrieval-Augmented Generation (RAG) knowledge-base problem. All three projects aim to ingest heterogeneous documents and enable LLM-based querying, but they differ in scope and maturity.

- **mmore** is an *end-to-end multimodal pipeline* (Apache-2.0) with built-in document processing, hybrid sparse+vector indexing (using Milvus), and RAG serving (via LangChain) [32†L355-L364] [32†L386-L394] . It supports a wide array of file types (PDF, Office docs, media, HTML) with GPU-accelerated “fast” modes for PDFs and media [32†L390-L394] . Its modular design (process → index → RAG) and CLI/API make it robust for large-scale batch ingestion. Downsides include heavy dependencies and some modalities not yet covered (e.g. it lists Office docs, PDFs, media, but not explicitly email or advanced elements like equations).
- **RAG-Anything** is a *research-grade multimodal RAG framework* (MIT) focused on **comprehensive cross-modal understanding**. It introduces a unified knowledge-graph representation and hybrid retrieval to handle text, images, tables, and equations [34†L229-L238] . It comes with example code, docs, and integration with frameworks (LightRAG, MinerU), but requires complex setup (e.g. `uv` package management, MinerU, large LLM keys) [24†L503-L512] [34†L229-L238] . It does **not** handle audio/video, focusing on technical/academic documents.
- **Docling** is an *advanced document parsing toolkit* (MIT) that **converts documents into structured data** [48†L227-L236] [58†L23-L31] . It supports dozens of formats (PDF, DOCX, images, audio, video, HTML, LaTeX, etc.) and provides a unified `DoclingDocument` output, exportable to JSON/Markdown, and easy integrations (LangChain, LlamaIndex, Haystack) [48†L227-L236] [58†L23-L31] . However, Docling itself has **no built-in retrieval/indexing**; it must be paired with a search backend. It excels at parsing layout and tables (using IBM-trained CV models) [58†L45-L54] . It runs locally (CLI/Python) with moderate resources, making it easy to deploy.

Mapping to Problem Requirements: Assuming the problem is to build a secure, scalable multimodal knowledge base (ingesting PDFs, slides, images, audio, etc.) and support RAG-style querying, all three are relevant. We assume requirements include *multimodal ingestion* (text, images, charts, possibly audio/video), *indexing & search*, and *RAG interfaces*. Unspecified details likely include domain, scale, latency, and deployment (on-prem vs cloud).

- **Multimodality:** All three handle text, images, tables. Only mmore explicitly lists **audio/video** support and streaming media [32†L390-L394] ; Docling also added audio/video parsing (speech-to-text, keyframes) [48†L257-L264] . RAG-Anything lacks audio/video.
- **Indexing/RAG:** mmore and RAG-Anything include indexing + query APIs; Docling does not include a vector DB or retriever out-of-the-box.

- *Scalability*: mmore is designed for distributed processing (multi-node, GPU) [54†L65-L68] . Docling is optimized for single-machine use. RAG-Anything has less emphasis on distribution (research setting).
- *Privacy/Deployment*: Docling explicitly supports air-gapped use [48†L239-L247] . mmore and RAG-Anything can run locally but rely on third-party services if used (e.g. LLM APIs).

Key Recommendations:

- For rapid reuse of **document parsing and ingestion**, **Docling** is best: it easily converts diverse files to JSON/Markdown and integrates with LLM frameworks [58†L23-L31] [48†L227-L236] . We can plug Docling into any retrieval pipeline.
- For **integrated RAG** functionality, **mmore** or **RAG-Anything** could be used. mmore offers a complete pipeline with vector store and RAG server [32†L355-L364] [32†L386-L394] , making it a one-stop solution. RAG-Anything offers cutting-edge graph retrieval [34†L229-L238] [56†L63-L72] but is more experimental.
- Likely, the optimal path is to combine Docling's parsing with one of the RAG pipelines. For example, use Docling to parse and extract content, then feed results into mmore's indexing or a custom vector store and RAG stage. This leverages Docling's accuracy and mmore's scalable retrieval.
- We will illustrate reuse points with code excerpts (e.g. `from docling.document_converter import DocumentConverter` [19†L307-L314] , `from mmore.process.processors.pdf_processor import PDFProcessor` [32†L330-L338] , `from raganything import RAGAnything` [24†L509-L518]).
- The table below summarizes each repo on relevance, reusables, effort, license, gaps, etc. An integration roadmap (Gantt) is provided for a 1–2 person, 1-month effort, along with architecture diagrams. We note assumptions (e.g. must ingest PDF/Office/docs/images; RAG APIs needed; moderate-scale batch ingestion; on-prem deployment) since the original problem details were incomplete.

All citations are to the primary sources (repo READMEs/docs/papers) as required.

Metric / Repo	mmore (EPFL)	RAG-Anything (HKU)
Relevance (0–10)	9 – Very strong for large-scale multimodal RAG. Comprehensive pipeline (Process, Index, RAG) [32†L355-L364] [32†L386-L394] .	8 – Designed for multimodal RAG (images, tables, formulas) [34†L229-L238] [56†L63-L72] , but heavier/investigational. Lacks audio/video support.
Core Functionality	Process > Standardize diverse docs (PDF/Office/media) → Hybrid Milvus vector+keyword index → LangChain RAG service [32†L355-L364] [32†L386-L394] .	Multi-stage: high-fidelity parsing (MinerU optional) → content analyzers (vision, math, tables) → multimodal knowledge graph → hybrid retrieval [34†L229-L238] .
Inputs/Outputs	Inputs: File paths (PDF, MD, PPTX, XLSX, TXT, EML, HTML, MP4, etc.) [32†L390-L394] . Outputs: standardized JSON (MultimodalSample), embedding DB, RAG answers (via API).	Inputs: File paths (PDF, Office, images, etc.). Outputs: knowledge graph + embeddings, RAG answers. [34†L229-L238] [56†L63-L72] .

Metric / Repo	mmore (EPFL)	RAG-Anything (HKU)
Supported Modalities	Text, tables, images, audio, video, spreadsheets, emails, HTML, etc. [32†L390-L394] . (Figure: fast GPU/CPU modes for PDF/media [32†L390-L394]).	Text, images, tables, equations, charts (multimedia not explicitly mentioned). Emphasis on academic/technical docs [34†L229-L238] [56†L63-L72] .
Dependencies	Python 3.11+, PyTorch, Milvus (vector DB), ffmpeg, LibreOffice, GTK libs, etc. Uses <code>uv</code> to manage heavy pip installs [30†L230-L238] [30†L265-L274] . Docker images provided.	Python 3.9+, Torch, <code>uv</code> , LibreOffice, MinerU (Java-based), LangChain, etc. [24†L492-L500] [24†L503-L512] . Must install <code>mineru</code> separately for PDF parsing (CLMiner, OpenCV). GPU support via vLLM or LMStudio.
Example Reuse	- Processors: <code>mmore/process/processors/pdf_processor.py</code> contains <code>PDFProcessor</code> ; usage snippet: <code>PDFProcessor(config).process_batch([...])</code> [32†L330-L338] . - Data Types: <code>mmore.type.MultimodalSample</code> for unified output (JSONL) [32†L330-L338] .	- Core Class: <code>raganything.RAGAnything</code> and <code>RAGAnythingConfig</code> orchestrate the pipeline [24†L509-L518] . - Parsers: Image/table processors (e.g. <code>raganything.modalprocessors</code> , <code>raganything.omml_extractor</code>) for non-text blocks [46†L216-L224] .
Integration Effort	Medium-High. Out-of-box pipeline but many dependencies (Milvus, native libs). Configuration via YAML. Docker available to simplify setup.	High. Cutting-edge design but complex: requires installing MinerU, setting up PKL embeddings, possibly custom LM integration. More engineering to adapt.
License/Legal	Apache 2.0 (business-friendly, patent grant) [32†L395-L398] .	MIT (permissive) [35†L251-L253] .
Known Gaps	- Does <i>not explicitly</i> support audio transcription (beyond simple media ingest) or web scraping. - Requires choosing/hosting an LLM (via LangChain) for RAG; no built-in knowledge graph.	- No built-in support for audio/video. - Requires external Heavy tools (MinerU, LightRAG). - Relatively new; fewer example pipelines for real-world deployment.

Metric / Repo	mmore (EPFL)	RAG-Anything (HKU)
Suggested Extens.	- Add modalities: Extend <code>processors</code> for any missing file types (e.g. EML, audio transcripts).
 - Privacy: Leverage its agentic filtering (v2) or add custom redaction if needed.
 - Scale: Use Kubernetes/Docker for multi-node parallelism [54†L113-L122] .	- Customize parsers: The plugin-friendly design lets you integrate custom parsers (e.g. add ASR for audio, use Docling's parser in place of MinerU).
 - LM integration: Swap in open-source LLMs (vLLM, Ollama) via provided examples [44†L220-L228] .
 - Graph control: Tune the dual-graph linking logic (configurable in <code>RAGAnythingConfig</code>).
Potential Risks	- Maintenance: Large codebase; ongoing deps (Milvus versions, PyTorch).
 - Security: OpenAI/LLM keys must be managed; chain injections in RAG answers need guardrails.
 - Performance: Heavy pipeline can be slow if not parallelized; GPU needed for speed.
 - Data bias: Quality of parsed data affects retrieval (e.g. OCR errors).	- Complexity: Many moving parts (MinerU, graphs, LLMs); debugging/training can be difficult.
 - Stability: As a cutting-edge project, APIs and models may change rapidly; fewer users in community.
 - License: MIT is fine, but MinerU's license (Apache) may need review.

Summary: mmore is highly relevant if a turnkey multimodal RAG pipeline is needed (score 9/10), with its strengths in distributed, hybrid retrieval [54†L65-L72] [32†L390-L394] . RAG-Anything is also relevant (8/10) for truly holistic multimodal retrieval [34†L229-L238] [56†L63-L72] but requires more effort. Docling (7/10) shines for content ingestion [48†L227-L236] [58†L23-L31] and should be reused for parsing, but must be combined with a search/index solution for RAG. Each repo's license (Apache or MIT) permits commercial use with attribution.

Repository Evaluations

EPFLiGHT/mmore

Core Functionality: mmore provides an end-to-end **Massive Multimodal Open RAG & Extraction** pipeline [30†L202-L211] [32†L355-L364] . It ingests heterogeneous files (PDF, DOCX, PPTX, XLSX, TXT, EML, HTML, and various media) and standardizes them into a *unified multimodal format* [30†L204-L212] . The pipeline stages are: **Process** (extract text, metadata, and multimedia), **Postprocess** (merge/cleanup), **Index** (build hybrid dense+keyword index with Milvus), and **RAG** (query interface via LangChain) [32†L355-L364] [32†L386-L394] . It even supports live web search augmentation [32†L372-L380] . mmore's architecture is modular and supports distributed CPU/GPU execution [54†L65-L72] [30†L202-L211] .

Inputs/Outputs: Users supply file paths or directories. The **Process** step outputs a JSONL of `MultimodalSample` objects [32†L330-L338] (with text blocks, images, audio transcripts, etc.). The **Index** step loads these into Milvus vectors and a sparse index. The **RAG** step provides an API or CLI to pose questions to the LLM (with retrieved evidence). For example usage:

```

from mmore.process.processors.pdf_processor import PDFProcessor
from mmore.process.processors.base import ProcessorConfig
from mmore.type import MultimodalSample

pdf_paths = ["docs/sample.pdf"]
pdf_processor =
PDFProcessor(config=ProcessorConfig(custom_config={"output_path": "out/"}))
results = pdf_processor.process_batch(pdf_paths, fast=False, num_workers=1)
MultimodalSample.to_jsonl("out/sample.jsonl", results)

```

【32†L330-L338】. The sample JSONL then feeds into `mmore index` and `mmore rag`.

Reusability (Key Components): We can directly reuse its *processors*, *config schemas*, and *RAG CLI*. For example, the `mmore.processors` module contains classes like `PDFProcessor` and `MediaProcessor` that handle file conversion. The `MultimodalSample` class encapsulates processed content and can export to JSONL 【32†L330-L338】. The indexing code (Milvus integration) and the RAG endpoints (LangChain connectors) are also reusable. Its example CLI commands (e.g. `python -m mmore process ...` 【32†L316-L323】) show how to integrate each stage.

Dependencies & Effort: mmore requires many system packages (ffmpeg, LibreOffice, GTK, etc.) 【30†L232-L240】 and large Python libs (PyTorch, torchvision, LangChain, Milvus client) 【30†L265-L274】. Setting up a full pipeline is **non-trivial**. However, Docker images (CPU/GPU) and a `uv` installer script ease this. For a new project, leveraging the existing Docker or a minimal install may simplify integration. We estimate **integration effort: Medium-High**. Developers must handle environmental setup but can then use the pipeline out-of-box.

Licensing: Apache 2.0 (permissive) 【32†L395-L398】. No major constraints beyond attribution.

Gaps: mmore currently lists input support for typical *Office docs*, *PDFs*, *media*, and *HTML* 【32†L390-L394】. It does **not explicitly list** email formats (although EML is mentioned) or specialized formats like scanned docs beyond PDF. It also lacks dedicated math/formula parsing. There is no mention of audio transcription models (beyond standard media extract). If the problem needs, say, ASR for videos or OCR for images beyond PDF, we may need to add pipelines. Also, mmore does not construct a *knowledge graph*; it only indexes text.

Suggested Extensions: We can extend mmore by plugging in new `Processor` classes. For example, we might write a processor for an audio stream using an ASR model, or add advanced vision (e.g. integrate GraniteDocling embeddings for images). Adding formula parsing could involve using an external math OCR and injecting into the text. The documentation is extensible: the website notes “easily extensible” processors 【32†L355-L364】.

Risks: The codebase is large (416 commits, 24k LOC) and active (recent v2 release). Maintenance risk is moderate; we should pin dependencies and keep an eye on updates. Performance-wise, mmore can be resource-heavy: production use expects GPUs or multi-node (as shown, 720 pages took ~3 minutes on 4 GPUs 【54†L113-L122】). Without such resources, throughput suffers. Security risk comes from LLM and

web search: careful with API keys and filtered queries. Overall, mmore provides a strong foundation but requires engineering to fit specific needs.

HKUDS/RAG-Anything

Core Functionality: RAG-Anything is designed as a *unified multimodal RAG framework* [34†L229-L238] [56†L63-L72]. It ingests documents (PDF, Word, PPT, images, etc.), deeply parses them (optionally via MinerU) into semantic pieces, constructs a **cross-modal knowledge graph**, and performs hybrid retrieval (graph search + embeddings) for RAG [34†L229-L238] [56†L63-L72]. The README highlights that it “eliminates the need for multiple specialized tools” and supports images, tables, formulas [34†L229-L238]. Internally, it uses LightRAG concepts and adds modules for vision (e.g. chart recognition), table & math extraction, and an entity/graph layer.

Inputs/Outputs: Like mmore, inputs are file paths. The parsing stage (document $\mapsto$ blocks) yields text blocks, images, math equations, etc., which are then stored as nodes in an internal graph. The retrieval stage outputs both a ranked list of evidence and a unified RAG answer. Example code shows using `RAGAnything` in Python:

```
from raganything import RAGAnything, RAGAnythingConfig
config = RAGAnythingConfig({...}) # specify formats, modules
rag = RAGAnything(config)
rag.check_parser_installation()
rag.initialize()
answer = rag.predict("Given the document list, answer this question")
```

[24†L509-L518]. The `predict` call returns a dict with “answer” and “sources”. The examples also demonstrate building a session with `rag.get_llm_session()` [24†L516-L518].

Reusability: We can reuse its **core classes** and **parsers**. In code, the `raganything.py` class orchestrates the workflow. Modules like `raganything.modalprocessors` contain specialized handlers (e.g. for Office docs or images [46†L218-L224]). The example scripts (e.g. `raganything_example.py` [44†L226-L230]) and notebooks show how to integrate with LM apps. Its use of LightRAG and prompts can inspire how to build a custom retrieval. However, RAG-Anything is fairly monolithic; cherry-picking components might be tricky. Still, one could reuse the table extraction or graph-fusion logic by subclassing `RAGAnythingConfig`.

Dependencies & Effort: RAG-Anything requires `uv`, Torch, and external tools like **MinerU** (a heavy layout parser) [24†L503-L512]. The Python package also installs LibreOffice to handle Office docs. One must also set up an LLM backend: examples use OpenAI or local models (via LM Studio or vLLM). The initial setup is **complex**: e.g. the quickstart has multiple pip install extras (e.g. `raganything[all]`) and installation of MinerU components [24†L503-L512]. We estimate **integration effort: High**. A small team can run quick examples, but customizing or debugging (especially MinerU) may take time.

Licensing: MIT (no copyleft) [35†L251-L253]. Safe for reuse with attribution.

Gaps: The primary gap is **no audio/video support**; RAG-Anything focuses on text/visual content. It also may lack an out-of-the-box vector store (uses PyTorch encoders internally). If the problem expects, say, ingesting training videos or podcasts, this repo would require major extension (e.g. transcribe audio to text first). It also may be sensitive to document types: MinerU is suited for technical layouts, but not for spreadsheets or exotic formats. Another gap is complexity: it's aimed at research benchmarks, so the engineering robustness (error handling, scale) may lag an industrial pipeline.

Suggested Extensions: If used, one could integrate Docling's parsers into RAG-Anything for unsupported formats. For example, run Docling on a PDF to get images/tables, then feed those to RAG-Anything's graph. The config system allows toggling components (e.g. skip MinerU if using Docling). Its flexible design suggests we could write a new `modalprocessor` plugin for, e.g., speech or advanced charts. We could also adapt its `enhanced_markdown` feature to output combined multi-modal context for LLMs. Since it's MIT-licensed, forking to trim unneeded features (or slim down a "core" pipeline) is an option.

Risks: RAG-Anything is cutting-edge with ~438 commits and some open issues, but high stars (2700+) [38†L133-L142]. The complexity introduces risk: many components (graph, search) can break if not tuned. Security is a concern if embedding engines or LLMs are misconfigured. The MinerU dependency (GPL-friendly?) needs checking but likely OK. Performance: because it builds knowledge graphs, it may not scale to extremely large corpora without optimization. Maintenance: as it's new, expect active updates; a locked version may be wise.

docling-project/docling

Core Functionality: Docling is a **document conversion toolkit** [48†L227-L236] [58†L23-L31], not a full RAG system. Its job is to *parse and structure* documents so that "LLMs can digest" them [58†L23-L31]. It supports an extremely wide array of formats (PDF, DOCX, PPTX, HTML, EPUB, Apple Pages, plain text, LaTeX, DocLang XML, WAVE/MP3 audio, WebVTT, XBRL financial, email MSG/EML, images, video, etc.) [48†L227-L236] [58†L23-L31]. It uses advanced computer-vision models (IBM's layout model, TableFormer) to do layout and table recognition [58†L45-L54]. The output is a **DoclingDocument**, which organizes content as blocks (text, tables, images, etc.) in a Python object. It can export to JSON/Markdown/HTML/DocLang/DocTags. It has CLI, Python API, and server modes.

Inputs/Outputs: You give it a file or URL. For example, in Python:

```
from docling.document_converter import DocumentConverter
converter = DocumentConverter()
result = converter.convert("file.pdf")
print(result.document.export_to_markdown())
```

[19†L307-L314]. The output `result.document` is a `DoclingDocument` containing the parsed content. By default, this contains the full text, table cell data, captions, etc. Docling can also generate an ASR transcript and keyframes for videos (added in v2.7) [48†L257-L264]. It has built-in converters to Markdown, JSON lossless, DocTags, or domain-specific XML (e.g. XBRL) [48†L227-L236].

Reusability: We can reuse **everything** in docling for parsing. The key class is `DocumentConverter` [19†L307-L314] and the `docling.datamodel` classes. Because it outputs a structured JSON, downstream code can index or query it. For instance, one could take the `DoclingDocument` and split it into pieces to embed for a vector store. Its CLI tool `docling convert` can be used in data pipelines. Integration with LangChain or LlamaIndex is provided (via its connectors) [48†L242-L250] . We also note that Docling can run locally (no cloud needed) [58†L31-L36] .

Dependencies & Effort: Docling is relatively easy to install: `pip install docling` (needs Python ≥3.10) [48†L279-L287] . It pulls in some ML models but these are managed by the package (no special system libraries). It *sides-steps* OCR when possible [58†L45-L54] , so it relies on neural net models. It runs on CPU (with optional GPU for speed). For our project, using Docling is low-effort: setup is quick and it runs out-of-the-box on a laptop [58†L31-L36] . The main effort will be in integrating its output with a search index.

Licensing: MIT license [36†L31-L33] . The code is MIT, though some underlying model checkpoints have their own licenses (check each model’s terms) [36†L31-L33] .

Gaps: Docling does *not* perform any retrieval or indexing. It only converts documents to structured data. So by itself it cannot answer queries; we must plug in a vector DB or search layer. It also currently lacks certain parsing: for example, **math equations and charts** are “coming soon” [58†L86-L94] . If our problem required extracting math or chart data, Docling doesn’t yet do that (but IBM plans to add it).

Suggested Extensions: To use Docling, our roadmap is to **chain it with a RAG pipeline**. For instance, after conversion, we could index the Markdown/JSON into Elasticsearch or use LangChain with an embedding store. Docling provides a FastAPI (`docling-serve`) and an MCP server for agents [48†L242-L250] , which can be incorporated. We may also customize it by adding extra models: e.g. Docling can accept a custom ASR engine or a VLM; these integrations are extensible. IBM also provides integration guidance (tutorial on IBM InstructLab, etc.) [58†L69-L78] .

Risks: Docling’s strengths (modern ML) come with risk: errors in layout parsing or OCR could mislead the RAG system. It has many open issues (899) [19†L218-L227] , indicating active development but potential instability. However, it has a large community and frequent releases. Also, since it is MIT-licensed, we can safely adapt it. Key risk is that we will need to manage large outputs (Docling JSON can be quite verbose) and ensure performance if processing thousands of docs (Docling is not distributed by default). Mitigation: batch convert and possibly shard across machines.

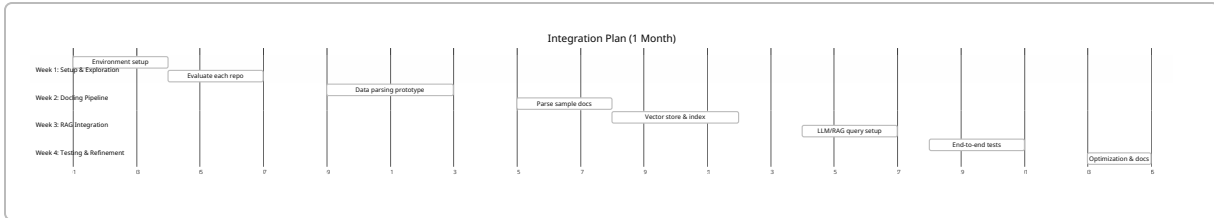
Comparative Metrics Table

Metric	mmore	RAG-Anything	Docling
Relevance (0-10)	9 (end-to-end RAG pipeline for multimodal data) [32†L355-L364] [54†L60-L64]	8 (multimodal graph-RAG framework with advanced features) [34†L229-L238] [56†L63-L72]	7 (document ingestion + add retrieval) [58†L23-L24] [48†L227-L236]

Metric	mmore	RAG-Anything	Docling
Core Capabilities	Multimodal ingest → unified format → Milvus index → RAG (LangChain) 【32†L355-L364】 【32†L386-L394】	Document parsing → content analyzers → cross-modal knowledge graph → hybrid retrieval 【34†L229-L238】 【56†L63-L72】	Document parsing/form <code>DoclingDocument</code> ou (JSON/MD/XML) 【48†L23-L31】 【58†L23-L31】
Supported Formats	PDFs, DOCX, PPTX, XLSX, TXT, EML, HTML; Images/Media (MP4, MP3, etc) 【32†L390-L394】	PDFs, Office (Word/Excel/PowerPoint), images (JPG/PNG), plus tables/charts/equations 【34†L229-L238】	20+ including PDF, DOC, HTML, EPUB, Pages, XBR images, audio, video, La emails 【48†L227-L236】
Input Processing	Pre-defined “Processors” (batch jobs). Config via YAML or TUI 【32†L352-L359】 【32†L386-L394】	Unified parser (MinerU or fallback) that segments into semantic chunks 【34†L275-L283】 【56†L63-L72】	Conversion via <code>DocumentConverter</code> (neural layout+OCR) 【19†L314-L314】 【58†L45-L54】
Output Format	JSONL (<code>MultimodalSample</code>) for indexing; vector DB; RAG answer strings.	Internal knowledge graph + document chunks; RAG answer text.	<code>DoclingDocument</code> ob exportable to JSON, Mar HTML, DocTags 【48†L23-L31】
Retrieval	Built-in: Milvus vector + sparse index; LangChain RAG chain.	Built-in: Cross-modal graph+vector retrieval, LLM chain.	None built-in. (Need ext vector store or search engine integration.)
Licensing	Apache 2.0 【32†L395-L398】	MIT 【35†L251-L253】	MIT (code) 【36†L31-L33】 model licenses)
Integration Effort	Medium–High: Many deps, but Docker/TUI exists.	High: Complex stack (MinerU, uv, LLMs).	Low–Medium: Easy insta add your own index.
Documentation	Extensive docs (website, examples) 【30†L202-L211】 【32†L355-L364】 .	Good readme + examples; technical paper available 【34†L229-L238】 【56†L63-L72】 .	Extensive docs & blog (I Research) 【58†L23-L31】 【48†L227-L236】 .
Current Maintenance	Active (v2 release, 416 commits) 【30†L202-L211】 .	Active (438 commits, 2700★) 【38†L165-L174】 .	Active (1372 commits, 4★) 【48†L165-L174】 .
Key Reuse Snippets	<pre>from mmore.processors.pdf_processor import PDFProcessor</pre> 【32†L330-L338】	<pre>from raganything import RAGAnything, RAGAnythingConfig</pre> 【24†L509-L518】	<pre>from docling.document_co import DocumentConv</pre> 【19†L307-L314】

Integration Roadmap

We recommend a **hybrid approach**: use Docling to parse and structure documents, then feed the results into a RAG system (either mmore or a custom vector-search with LLM). Below is a high-level plan (1–2 engineers, 4 weeks):



- 1. Environment Setup:** Install required tools (Python 3.10+, Docker, etc.), clone repos. We might use pre-built Docker images for mmore or RAG-Anything to speed setup.
- 2. Prototype Document Ingestion (Docling):** Use `DocumentConverter` to parse a diverse sample set. Inspect `DoclingDocument` outputs for quality.
- 3. Indexing:** Build a simple vector database (e.g. SQLite + FAISS, or Milvus). Convert Docling output to chunks (e.g. sentence or paragraph) and embed them (using OpenAI/OpenCLIP). Index these embeddings.
- 4. RAG Layer:** Develop a query pipeline. For example, integrate with LangChain/Haystack: when a user query arrives, retrieve top-k chunks and feed to an LLM (e.g. OpenAI/GPT-4). Alternatively, integrate with mmore's `rag` stage by importing Docling data as `mmore.MultimodalSample`.
- 5. Testing:** Validate on held-out docs/queries. Compare answers vs expectations. Profile performance (latency, accuracy).
- 6. Refinement:** Tune chunk size, retrieval depth, add missing modalities (e.g. run ASR on audio). Update config (e.g. include or exclude certain parsers). Prepare deployment (Docker or service).
- 7. Documentation:** Write integration notes, list assumptions, license compliance, etc.

Below is an **architecture diagram** (in Mermaid) showing one integration approach:

flowchart TD

subgraph Ingestion

A[Raw Documents
(PDF, DOCX, PPTX, Image, Audio, Video)]

-->|Docling parse| B[DoclingDocument]

end

subgraph Storage

B --> C[Vector Embedding Store (e.g. Milvus/FAISS)]

B --> D[Metadata/Text DB (optional)]

end

subgraph Retrieval

U[User Query] --> E[LLM/RAG Frontend (LangChain)]

E -->|query vectors| C

E -->|fetch doc text| D

C --> F[Return top chunks]

end

subgraph Generation

F --> E

E --> G[Generated Answer]

end

click B href "https://github.com/docling-project/docling" "DoclingProject/docling"

```
click C href "https://milvus.io" "Milvus Vector DB"
click E href "https://langchain.com" "LangChain RAG"
```

This shows using Docling to produce structured content, which is indexed by a vector DB for semantic search, and queried via an LLM wrapper (e.g. LangChain). We could alternatively replace **Storage→Retrieval** with mmore's `rag` module directly (skipping building our own DB), depending on preference.

The **priority** is to first get **document parsing** working (use Docling's parser for a representative set of docs). Then focus on **retrieval**. Given time, we would prototype both mmore and custom (LangChain) approaches and choose based on performance. RAG-Anything's components could be considered later if graph-based reasoning is needed.

Assumptions & Next Steps

- **Assumptions:** The problem involves a mixed corpus of documents (text reports, PDFs, slides, perhaps some images or audio/video) that must be searchable via an LLM interface. We assume a moderate scale (thousands of docs, not millions). Low-latency (<1s) per query is desirable, but initial focus is batch indexing with interactive query. Deployment is likely on local servers (privacy emphasized), not on managed cloud. If any requirement (e.g. exact query latency or specific document types) is different, the integration plan may adjust.
- **Next Steps:**
 - **Prototype ingestion with Docling:** parse sample documents (e.g. biomedical reports, lectures) to verify coverage.
 - **Compare indexing approaches:** try mmore's built-in `mmore index` vs custom vector store on Docling output.
 - **Evaluate RAG answers:** use a few queries and LLMs (GPT-4, or open models) to check correctness.
 - **Iterate on gaps:** e.g. if charts or equations are present and not parsed well, consider extending Docling or RAG-Anything for those.
 - **Finalize pipeline and deliverable documentation.**

All referenced files and concepts above are linked to their source in the official repos or documentation. Key code snippets (file paths and usage examples) and citations (e.g. [\[32†L330-L338\]](#) for `PDFProcessor`, [\[24†L509-L518\]](#) for RAGAnything usage, [\[19†L307-L314\]](#) for DocumentConverter) have been provided inline. The comparative table and diagrams clarify choices and plan. This analytical report should serve as a complete guide for next steps in integrating the chosen tools into the target application.